

SISTEMAS OPERATIVOS

Herramientas de Sincronización

Mecanismos para garantizar el acceso ordenado a datos compartidos y mantener la consistencia en la ejecución concurrente.

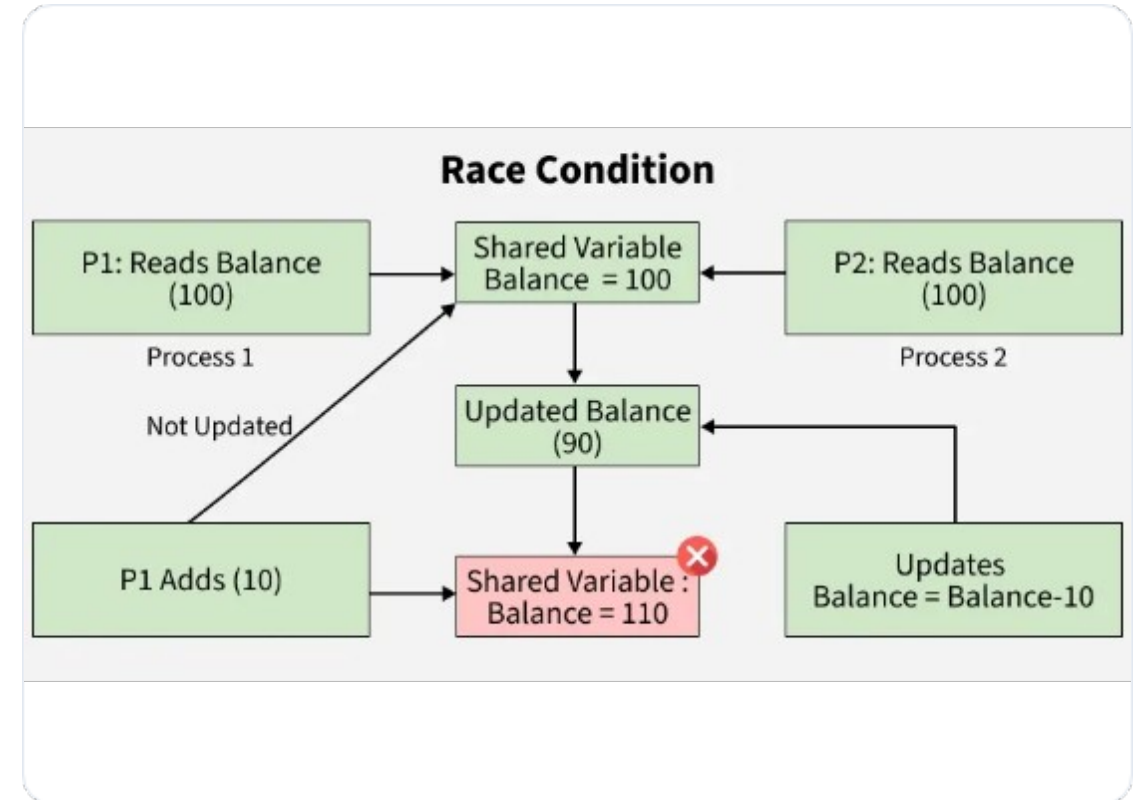
CONDICIÓN DE CARRERA

El problema de la concurrencia

El acceso concurrente a datos compartidos puede resultar en inconsistencia de datos.

Ejemplo (fork): Dos procesos solicitan un PID al mismo tiempo. Sin sincronización, el núcleo podría asignar el mismo identificador a ambos.

La inconsistencia requiere mecanismos que aseguren la ejecución ordenada de procesos cooperativos.



LA SECCIÓN CRÍTICA

Considere un sistema de n procesos. Cada uno tiene un segmento de código llamado **Sección Crítica** donde accede a recursos comunes.



Sección de Entrada

El proceso solicita permiso para entrar.



Sección Crítica

Ejecución exclusiva sobre datos compartidos.



Sección de Salida

Libera el acceso para otros procesos.

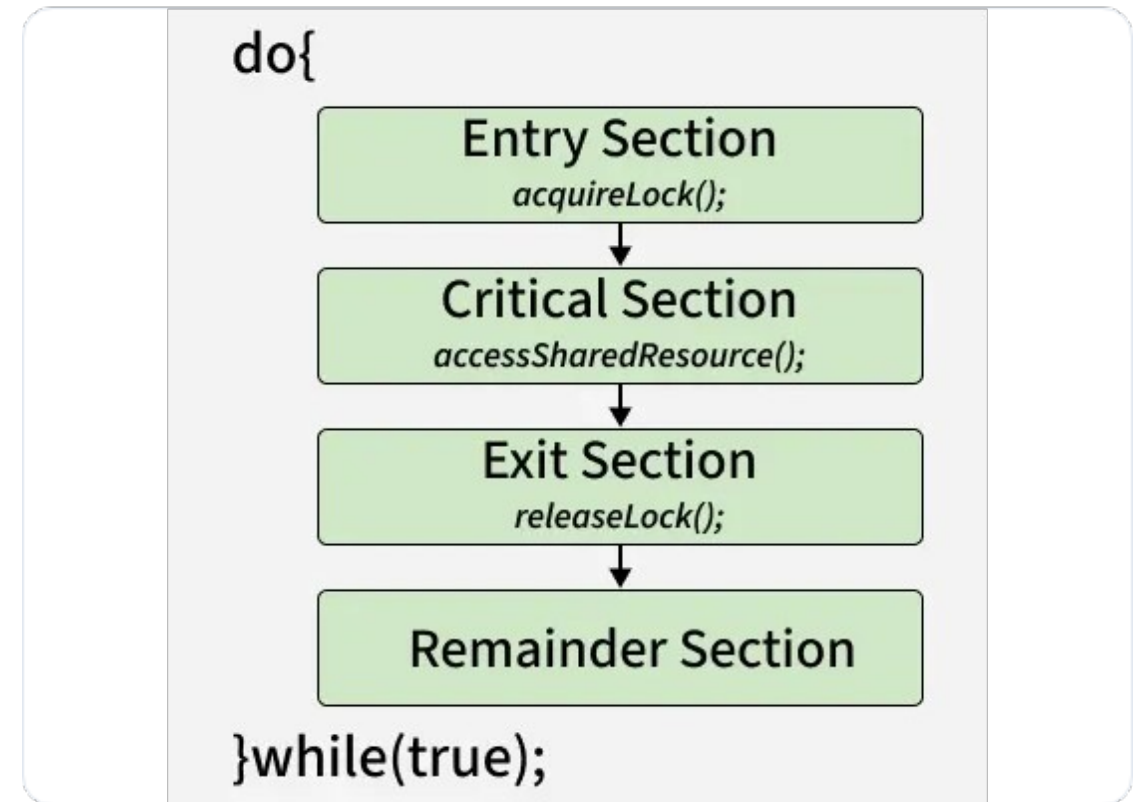


Sección Restante

Código no crítico del proceso.

REQUISITOS DE SOLUCIÓN

- 🛡️ **Exclusión Mutua:** Si el proceso P_i está en su sección crítica, ningún otro puede estar en la suya.
- ▶▶ **Progreso:** Si nadie está en la sección crítica, la decisión de quién entra no puede posponerse indefinidamente.
- 🕒 **Espera Limitada:** Existe un límite en el número de veces que otros pueden entrar antes de que se conceda una solicitud pendiente.



SOLUCIÓN DE PETERSON

Algoritmo de Software

Solución clásica para dos procesos basada en carga/almacenamiento atómicos.

```
flag[i] = true;  
turn = j;  
while (flag[j] && turn == j);  
// sección crítica
```

Arquitectura Moderna

Peterson **no garantiza** funcionamiento en hardware moderno debido al reordenamiento de instrucciones por compiladores y procesadores.

Solución: Uso de Barreras de Memoria.

SOPORTE DE HARDWARE



Instrucciones Atómicas

Hardware que permite probar y modificar el contenido de una palabra (*Test-and-Set*) o intercambiar dos valores (*Compare-and-Swap*) sin interrupción.



Variables Atómicas

Proporcionan actualizaciones atómicas sobre tipos de datos básicos como enteros y booleanos. Bloque de construcción para otras herramientas.



Barreras de Memoria

Instrucciones que obligan a que los cambios en la memoria se propaguen a todos los procesadores en sistemas de multiprocesamiento.

CERROJOS MUTEX

1

Variable Booleana

La herramienta más simple

Mecanismo basado en cerrojos (*locks*). Un proceso debe **adquirir** el cerrojo antes de la sección crítica y **liberarlo** después.

⚙️ **Spinlock:** Este tipo de cerrojo requiere "espera activa" (busy waiting), lo cual consume ciclos de CPU inútilmente si el bloqueo es largo.

SEMÁFOROS

Sincronización Avanzada

Variable entera **S** accedida mediante dos operaciones atómicas: wait() (P) y signal() (V).

S. Contador: Dominio de valores sin restricciones (recursos con múltiples instancias).

S. Binario: Solo 0 y 1 (similar a un Mutex).

Operación wait(S)

```
while (S <= 0); // espera activa  
S--;
```


SEMÁFOROS SIN ESPERA ACTIVA

Para evitar el desperdicio de CPU, los semáforos pueden implementar una cola de espera con dos operaciones del sistema:

Operación	Acción del Sistema	Efecto sobre el Proceso
<code>block()</code>	Coloca al proceso en la cola de espera del semáforo.	El proceso se suspende (estado de espera).
<code>wakeup(P)</code>	Mueve al proceso de la cola de espera a la cola de listos.	El proceso puede reanudar su ejecución.

Dato clave: El valor del semáforo puede volverse negativo, indicando cuántos procesos están bloqueados esperando el recurso.

MONITORES

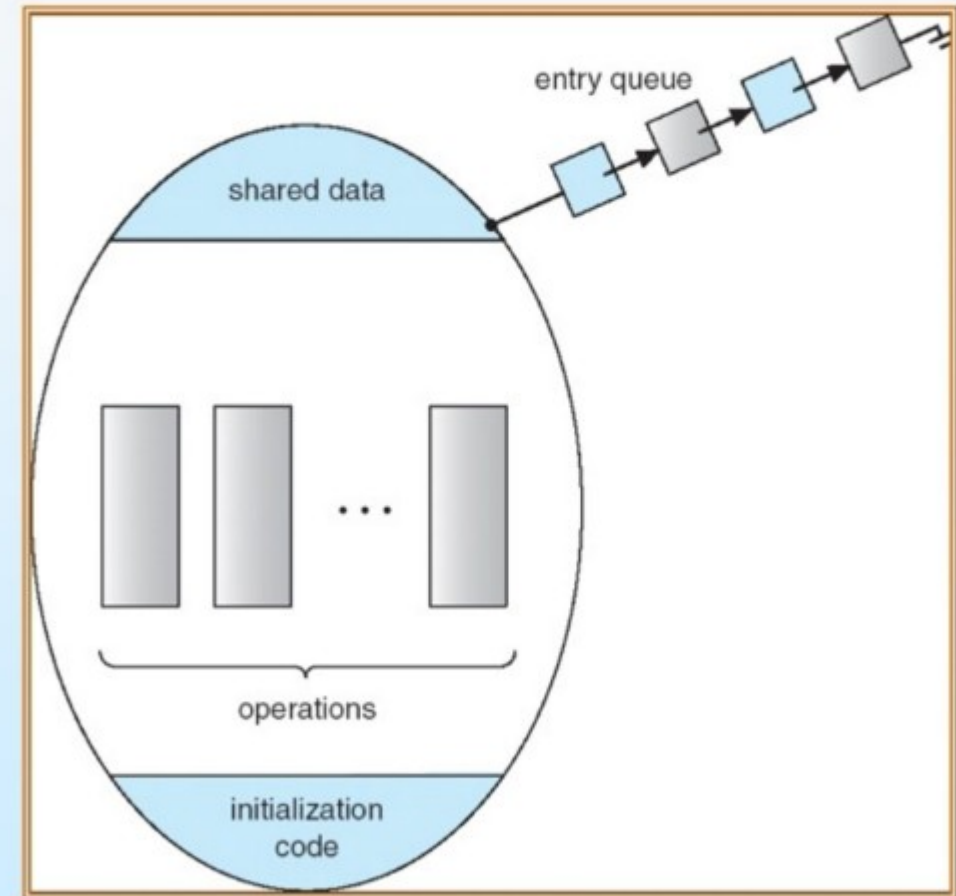
Abstracción de Alto Nivel

Tipo de dato abstracto que garantiza que solo un proceso a la vez esté activo dentro del monitor.

☒ **Variables de Condición:** Permiten que los procesos esperen eventos específicos (`x.wait()`, `x.signal()`).

🔧 Se suelen implementar utilizando semáforos internamente.

Schematic view of a Monitor

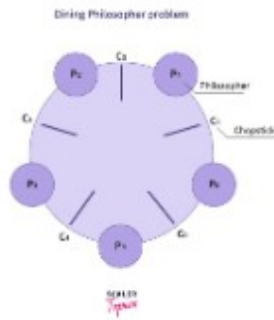


PROBLEMAS DE VIVACIDAD



Interbloqueo (Deadlock)

Dos o más procesos esperan indefinidamente por un evento que solo puede ser causado por otro de los procesos en espera.



Hambruna e Inversión

Hambruna: Un proceso nunca es retirado de la cola del semáforo.

Inversión de Prioridad: Un proceso de alta prioridad espera por uno de baja prioridad que tiene un recurso.

SISTEMAS OPERATIVOS

Ejemplos de Sincronización y Problemas Clásicos

Modelado de problemas clásicos de concurrencia y revisión de APIs de sincronización en sistemas reales.

ESTRUCTURA DEL PRODUCTOR Y CONSUMIDOR

⚙️ Código del Productor

```
while (true) {  
    // produce un elemento  
    wait(empty);  
    wait(mutex);  
    // añade al buffer  
    signal(mutex);  
    signal(full);  
}
```

🏠 Código del Consumidor

```
while (true) {  
    wait(full);  
    wait(mutex);  
    // remueve del buffer  
    signal(mutex);  
    signal(empty);  
    // consume el elemento  
}
```

PROBLEMA DE LOS LECTORES Y ESCRITORES

Variaciones de Acceso

Una base de datos o archivo compartido entre múltiples procesos concurrentes:

Lectores: Solo leen; pueden acceder varios de forma simultánea.

Escritores: Actualizan datos; requieren acceso exclusivo total.

Estructuras Base: Un entero `read_count` (cuenta lectores activos) y semáforos `mutex` (protege el contador) y `rw_mutex` (exclusión para el escritor).

Prioridades (Variaciones)

Primera variación: Ningún lector espera a menos que un escritor tenga permiso de usar el recurso (puede causar hambruna al escritor).

Segunda variación: Si un escritor está listo, realiza la escritura lo antes posible.

FILÓSOFOS COMENSALES

Modelado del Problema

Cinco filósofos pasan la vida pensando y comiendo. Comparten una mesa circular con 5 platos y 5 palillos aislados.

Para comer, un filósofo debe adquirir obligatoriamente **ambos palillos contiguos** (izquierdo y derecho).

Cada palillo se modela de forma básica como un semáforo independiente: semaphore chopstick[5];

Riesgo Máximo: Deadlock

Si los cinco filósofos se sientan al mismo tiempo y toman su palillo izquierdo simultáneamente, el sistema cae en un **Interbloqueo (Deadlock)**.

Ninguno podrá tomar el derecho y todos morirán de inanición.

SINCRONIZACIÓN EN EL NÚCLEO LINUX

Linux provee diferentes mecanismos dependiendo de la arquitectura y la duración del bloqueo:



Mecanismos Atómicos

Operaciones del tipo más simple sobre enteros (`atomic_t`) sin penalización de contexto.



Spinlocks

Utilizados para bloqueos de muy corta duración en multiprocesadores. El hilo realiza espera activa.



Mutex y Semáforos

Para tareas largas. Si el recurso está retenido, el proceso se suspende en una cola de espera.

SINCRONIZACIÓN EN WINDOWS

Objetos de Despacho

Windows sincroniza hilos utilizando **Dispatcher Objects**. Estos objetos pueden estar en dos estados legítimos:

Señalizado (Signaled): El recurso está disponible, un hilo en espera es despachado.

No Señalizado (Non-signaled): El recurso está ocupado, el hilo se bloquea.

Tipos de Objetos

Engloba Mutexes, Semáforos, Eventos y Temporizadores.

Los **Eventos** actúan de forma similar a variables de condición para notificar a hilos durmientes.

En sistemas multiprocesador se usan spinlocks adaptativos para secciones críticas cortas.

API POSIX (PTHREADS)

La API POSIX es estándar en sistemas UNIX/Linux y está disponible a nivel de usuario:

```
// Gestión de Mutex
pthread_mutex_t mutex;
pthread_mutex_init(&mutex, NULL);

pthread_mutex_lock(&mutex);
/* Sección Crítica */
pthread_mutex_unlock(&mutex);
```

```
// Semáforos POSIX
sem_t sem;
sem_init(&sem, 0, 1);

sem_wait(&sem);
/* Recurso Compartido */
sem_post(&sem);
```

SINCRONIZACIÓN EN JAVA

Monitores Incorporados

Cada objeto en Java posee de forma intrínseca un cerrojo de monitor único asociado a él.

Si un método se declara con la palabra reservada `synchronized`, el hilo debe adquirir el cerrojo del objeto para ejecutarlo.

Provee los métodos nativos `wait()`, `notify()` y `notifyAll()`.

```
public synchronized void incrementar() {  
    contador++;  
}  
  
public synchronized void esperarDato() {  
    while(!disponible) wait();  
}
```

ENFOQUES ALTERNATIVOS

Memoria Transaccional

Bloque de operaciones de lectura/escritura en memoria atómicas (`atomic{S}`). Si una operación falla, se realiza un rollback completo.

OpenMP

Directivas de compilador para paralelismo en C/C++. La instrucción `#pragma omp critical` convierte automáticamente un bloque en sección crítica.

Lenguajes Funcionales

Entornos como Erlang o Scala no mantienen estados mutables. Al ser las variables inmutables, se eliminan nativamente las condiciones de carrera.

SISTEMAS OPERATIVOS

Bloqueos Mutuos (Deadlocks)

Análisis de la caracterización del sistema, estrategias de prevención, evitación y recuperación de estados de interbloqueo.

El Modelo del Sistema

Composición de Recursos

El sistema consta de diversos tipos de recursos ($R_1, R_2, \dots, R_m$) como ciclos de CPU, espacio de memoria y dispositivos de E/S.

Cada tipo R_i posee W_i instancias operando en un ciclo de tres pasos:

-] **Solicitud:** El proceso pide el recurso.
- ⚙️ **Uso:** El proceso opera con el recurso.
- ↳ **Liberación:** El proceso devuelve el recurso.



Recursos del Sistema

Un bloqueo mutuo ocurre cuando cada proceso en un conjunto espera un evento que solo puede ser causado por otro proceso del mismo conjunto.

Caracterización de Deadlock

Un bloqueo mutuo puede surgir si las siguientes cuatro condiciones se cumplen simultáneamente:



Exclusión Mutua

Solo un hilo a la vez
puede usar un recurso
no compartible.



Retención y Espera

Un hilo retiene un
recurso mientras
espera por otros.



Sin Expropiación

Los recursos no se
pueden quitar a la
fuerza.



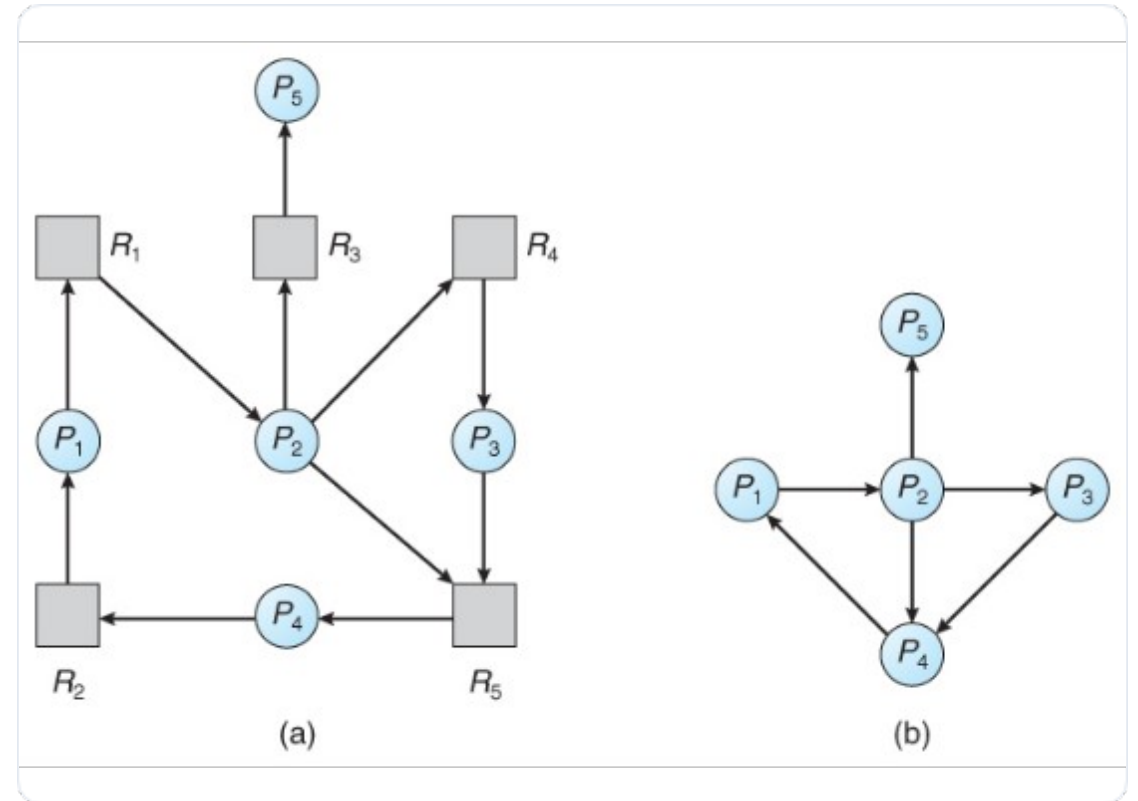
Espera Circular

Existe una cadena
cerrada de hilos
esperando recursos.

Visualización mediante Grafos

- **Vértices (V):** Particionados en T (Hilos) y R (Recursos).
- ➔ **Arista de Solicitud:** $T_i \rightarrow R_j$ indica que el hilo T_i ha solicitado una instancia de R_j .
- ⬅ **Arista de Asignación:** $R_j \rightarrow T_i$ indica que una instancia de R_j se ha asignado a T_i .

Si el grafo no contiene ciclos, no existe bloqueo mutuo en el sistema.



Análisis de Ciclos

Hechos Críticos

La presencia de un ciclo es necesaria pero no siempre suficiente para un bloqueo si hay múltiples instancias por recurso.

- ✓ Instancia única: Ciclo $\Leftrightarrow$ Deadlock.
- ⚠ Múltiples instancias: Ciclo $\Leftrightarrow$ Posibilidad de Deadlock.



Estrategias de Manejo

Prevención

Asegura que al menos una de las cuatro condiciones necesarias nunca se cumpla. Es el método más restrictivo.

Evitación

El sistema requiere información previa de las necesidades de recursos para tomar decisiones dinámicas seguras.

Detección

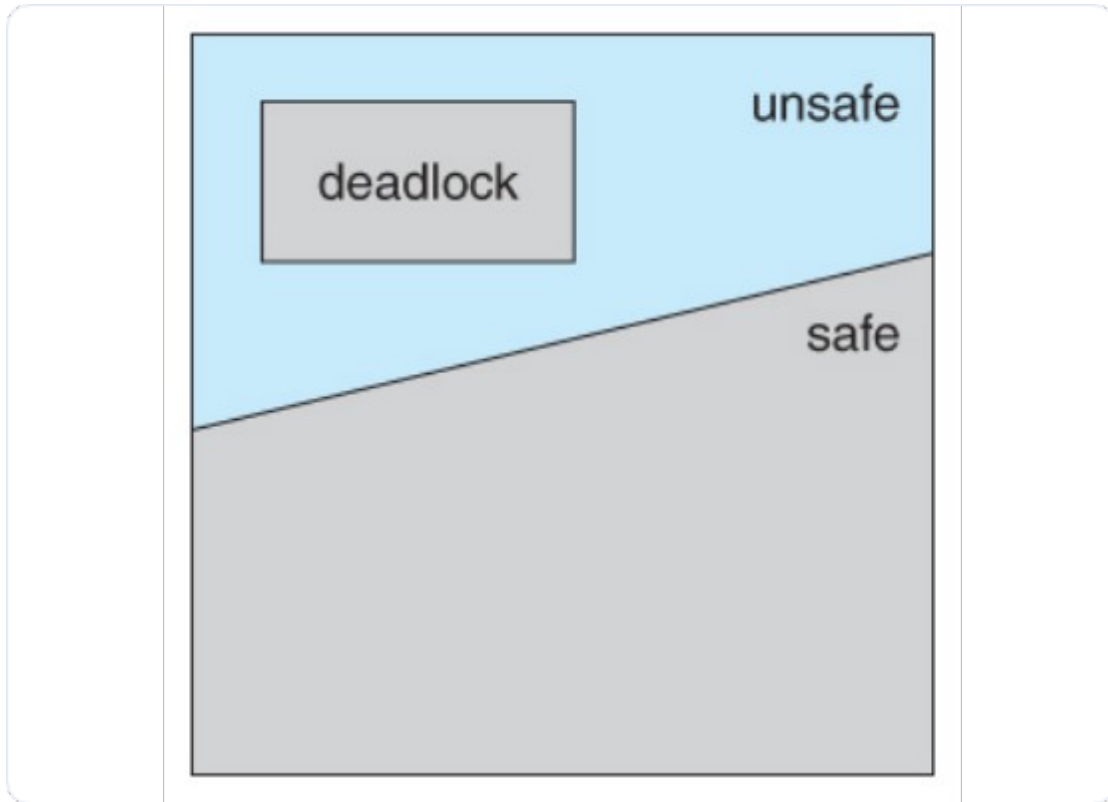
Permite que el sistema entre en deadlock, lo detecta periódicamente y aplica algoritmos de recuperación.

Nota: Muchos sistemas operativos modernos simplemente ignoran el problema (Enfoque del Avestruz).

Invalidación de Condiciones

Condición	Estrategia de Prevención
Exclusión Mutua	No requerida para recursos compartibles (ej. archivos de solo lectura).
Retención y Espera	Solicitar todos los recursos al inicio o solo cuando no tenga ninguno asignado.
Sin Expropiación	Si un proceso solicita un recurso no disponible, libera todos sus recursos actuales.
Espera Circular	Imponer un orden numérico total para la solicitud de tipos de recursos.

Evitación y Estados Seguros



Estado Seguro

Un sistema está en estado seguro si existe una secuencia de todos los hilos donde cada uno puede satisfacer sus necesidades con los recursos disponibles.



Seguro: Garantía de no deadlock.



Inseguro: Riesgo de deadlock; no se puede garantizar la seguridad.

Algoritmo del Banquero

Diseñado para múltiples instancias de recursos. Cada proceso declara su uso máximo *a priori*.

Estructura de Datos:

- Available[m] - Disponibles
- Max[n,m] - Demanda Máxima
- Allocation[n,m] - Asignados
- Need[n,m] - Necesidad Restante

$$\text{Need}_{ij} = \text{Max}_{ij} - \text{Allocation}_{ij}$$

Verificación de Seguridad

Antes de asignar, el sistema simula el estado resultante. Solo si el nuevo estado es "seguro", se otorga el recurso.

Si es inseguro, el hilo debe esperar aunque el recurso esté libre físicamente.

Detección de Bloqueos

Grafos de Espera (Wait-for)

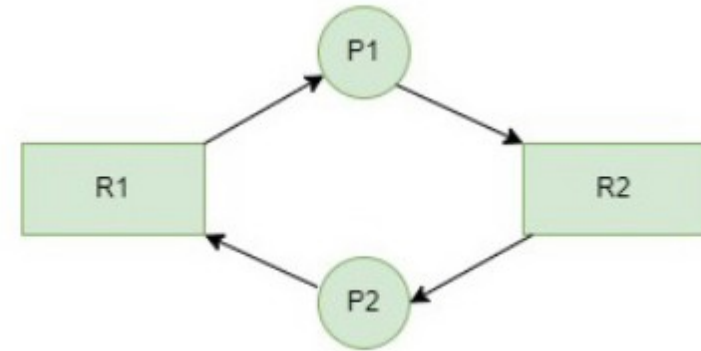
Nodos representados únicamente por hilos. Una arista $T_i \rightarrow T_j$ existe si T_i espera un recurso que T_j retiene.



Invoca periódicamente un algoritmo de búsqueda de ciclos.



Complejidad de $O(n^2)$ operaciones para n vértices.



Recuperación del Sistema



Terminación de Procesos

Abortar todos los hilos bloqueados.

Abortar uno por uno hasta romper el ciclo (alto costo).



Expropiación de Recursos

Víctima: Elegir el proceso con menor costo.

Rollback: Volver a un estado seguro anterior.

Starvation: Evitar elegir siempre al mismo.

Criterios de Aborto: Prioridad del hilo, tiempo ejecutado vs. tiempo para terminar, recursos consumidos vs. recursos necesarios.

Problema de los filósofos

Imagina que hay 5 filósofos sentados alrededor de una mesa redonda. Su vida se resume en hacer solo dos cosas: pensar y comer.

En el centro de la mesa hay un gran tazón de arroz (o espagueti), y se dispone de 5 palillos individuales en la mesa, uno entre cada par de filósofos adyacentes.

Aquí está el dilema:

- Para poder comer, un filósofo necesita obligatoriamente usar 2 palillos al mismo tiempo: el que está a su izquierda y el que está a su derecha.
- Cuando un filósofo tiene hambre, intenta tomar sus dos palillos vecinos, uno a la vez. Si sus vecinos los están usando, debe esperar.
- Cuando termina de comer, suelta ambos palillos y vuelve a pensar.

Representación gráfica

